

So verwendet ihr opencode

Getränkeautomat, Story für Story: Spec, Szenarien, Test, Code, Review

opencode sitzt zwischen dir und dem LLM

Du

- gibst ein Ziel vor
- prüfst jedes Diff
- entscheidest, was gilt

opencode

- Kontext: AGENTS.md, Dateien, Verlauf
- Tools: lesen, suchen, ändern, bash
- Schleife: bis das LLM fertig meldet
- Grenzen: Rechte und Schrittlimit

LLM

- liest den Kontext
- erzeugt Text: Antwort oder Tool-Aufruf
- sieht nur, was opencode ihm zeigt

Das LLM entscheidet, opencode handelt. Zusammen sind sie ein Agent.

Ein Agent verfolgt ein Ziel mit Werkzeugen

Ziel

- bekommt ein Ergebnis vorgegeben, keine Einzelschritte

Werkzeuge

- liest, ändert und führt aus, in deinem Projekt

Schleife

- handelt, sieht das Ergebnis, entscheidet neu

Grenzen

- Rechte, Schrittlimit, ein Mensch, der prüft

Agent = LLM + Kontext + Tools + Schleife. Ein Chatbot antwortet. Ein Agent handelt, prüft das Ergebnis und macht weiter.

Bedienung

Was	Wie
starten	im Projektordner: opencode
Rolle wechseln	Tab: build, plan, spec-autor, test-autor, implementierer, reviewer
Datei in den Kontext	@src/main/java/.../VendingMachine.java im Text
letzte Änderung zurück	/undo
eigene Commands	/spec 1, /akzeptanztest 1, /implementiere Ein Getränk wählen, /review
neue Session	/new, vor jedem Command und je Szenario

Was opencode mitbringt und was nicht

Bringt es mit

- Dateien lesen, suchen, ändern
- Befehle ausführen, Ergebnis lesen
- Rechte: allow, ask, deny
- AGENTS.md als Dauerkontext

Bringt es nicht mit

- die Reihenfolge der Schritte
- den Maßstab für gut genug
- das Fertig-Kriterium
- das Fachwissen zum Automaten

Das Repo ist der stärkste Prompt: AGENTS.md, Commands, Skills, Gates liegen im Projekt.

Die Bausteine im Projekt

Baustein	Was er tut	Wann im Kontext
AGENTS.md	Befehle, Struktur, Arbeitsweise, Code-Regeln	immer, jede Session
features/AGENTS.md	Regeln für Szenarien	sobald eine Feature-Datei gelesen wird
Commands	Prompt-Vorlage mit Spec, Glossar, mvn-Ausgabe	wenn du /spec, /akzeptanztest, /implementiere, /review tippst
Skills	Checklisten: EARS, elf Regeln, TDD, Clean Code	der Command fügt sie ein
Agents	Rolle mit Rechten und Schrittlimit	der Command wählt die Rolle
Gates	mvn -q verify: Tests, Checkstyle, ArchUnit, JaCoCo	vor jedem Fertig

Die Pipeline je Story

Story

Karte aus
specs/stories.md

/spec

Regeln in EARS,
specs/<nr>-
<name>/spec.md

/akzeptanztest

Szenarien in Gherkin,
Schritte in Java

/implementiere

je Szenario: roter Test,
Code, aufräumen, mvn
-q verify

/review

Befunde mit
Datei:Zeile,
Smell,
Vorschlag

Du prüfst nach jedem Schritt: Stimmen die Regeln? Erfüllen die Szenarien die elf Regeln? Verstehst du das Diff? Welche Befunde nimmst du an?

Spec Driven Development

Erst die Regel, dann das Beispiel, dann der Code.
Die Feature-Datei ist der ausführbare Teil der Spec.

EARS WHILE es vor 16:00 Uhr ist, WHEN der Kunde Bier wählt,
 SHALL der Automat keine Dose ausgeben und "Kein Bier vor 4" melden.

Gherkin Szenario: Vor 16 Uhr fällt kein Bier
 Angenommen es ist 15:59 Uhr
 Und ich habe 2 € eingeworfen
 Wenn ich Bier wähle
 Dann ist das Ausgabefach leer
 Und der Automat meldet "Kein Bier vor 4"

Elf Regeln für Akzeptanztests

1. Ein Szenario, ein Verhalten: genau ein Wenn, keins bei einer Regel ohne Auslöser.
2. Deklarativ: was, nicht wie. Keine Knöpfe, Klicks, IDs, HTTP.
3. Fachsprache des Kunden: Namen statt Nummern.
4. Angenommen ist Zustand, Wenn ist Ereignis, Dann ist von außen sichtbar: Dose, Guthaben, Meldung, Preis, Bestand.
5. Konkrete Beispiele mit Grenzwerten, nur relevante Details.
6. Der Titel nennt die Regel, nicht den Ablauf.
7. Unabhängig und wiederholbar: Zeit und Zufall gestellt.
8. Vorhandene Schritte wiederverwenden, neue nur bei fehlendem Wort.
9. Jedes Kriterium: Normalfall, Randfall, Fehlerfall.
10. Höchstens sechs Schritte, Grundlage sparsam.
11. Schritte sprechen mit dem Automaten, nicht mit der Oberfläche.

Vorher und nachher: Story 1

Vanilla opencode, zwei Versuche

- Versuch 1, 57 Sekunden: mvn -q verify grün. 16 Zeilen in VendingMachine.java, kein Test, keine Spec.
- Nebenbei takeCoins und coinReturn gebaut, die keine Story verlangt.
- Versuch 2, 10 Sekunden: Dateipfad außerhalb des Projekts erfunden, Rechteabfrage abgelehnt, aufgehört.

Mit Pipeline, vier Minuten

- /spec: eine EARS-Regel, keine offenen Fragen.
- /akzeptanztest: zweites Szenario „Auch Bier fällt gratis“, beide rot.
- /implementiere: roter Unit-Test zuerst, dann neun Zeilen Code, verify grün.
- /review: zwei Befunde mit Datei:Zeile, zum Bestand und zum Szenariotitel.

Gleiches Modell, gleiche Story. Der Unterschied: Regeln, Szenarien, Test und Review, jedes Mal in dieser Reihenfolge.

Deine Verantwortung

- Du reviewst jedes Diff. Der Agent liefert Vorschläge, du lieferst Code.
- Wer eine Regel nicht formulieren kann, kann sie auch nicht vom Agenten verlangen.
- Kleine Schritte: ein Szenario, eine Session, ein Commit.
- Hängt der Agent: Szenario kleiner schneiden, /implementiere neu starten, scripts/bis-gruen.sh.
- `mvn -q verify` entscheidet. Nicht das Gefühl, nicht der Agent.